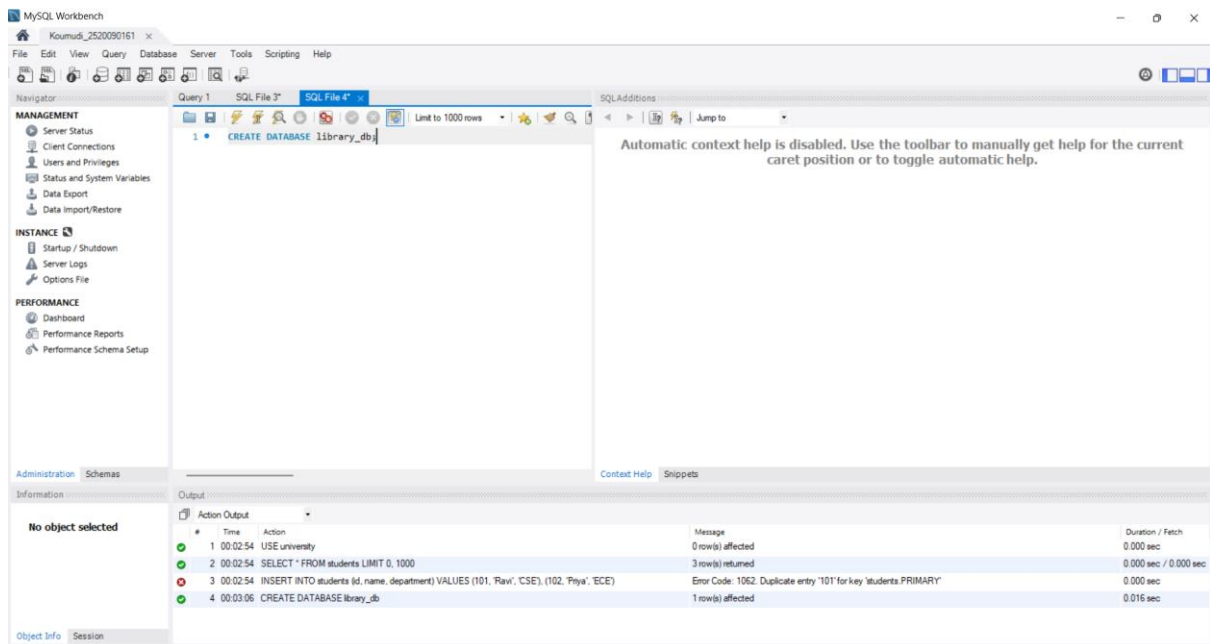
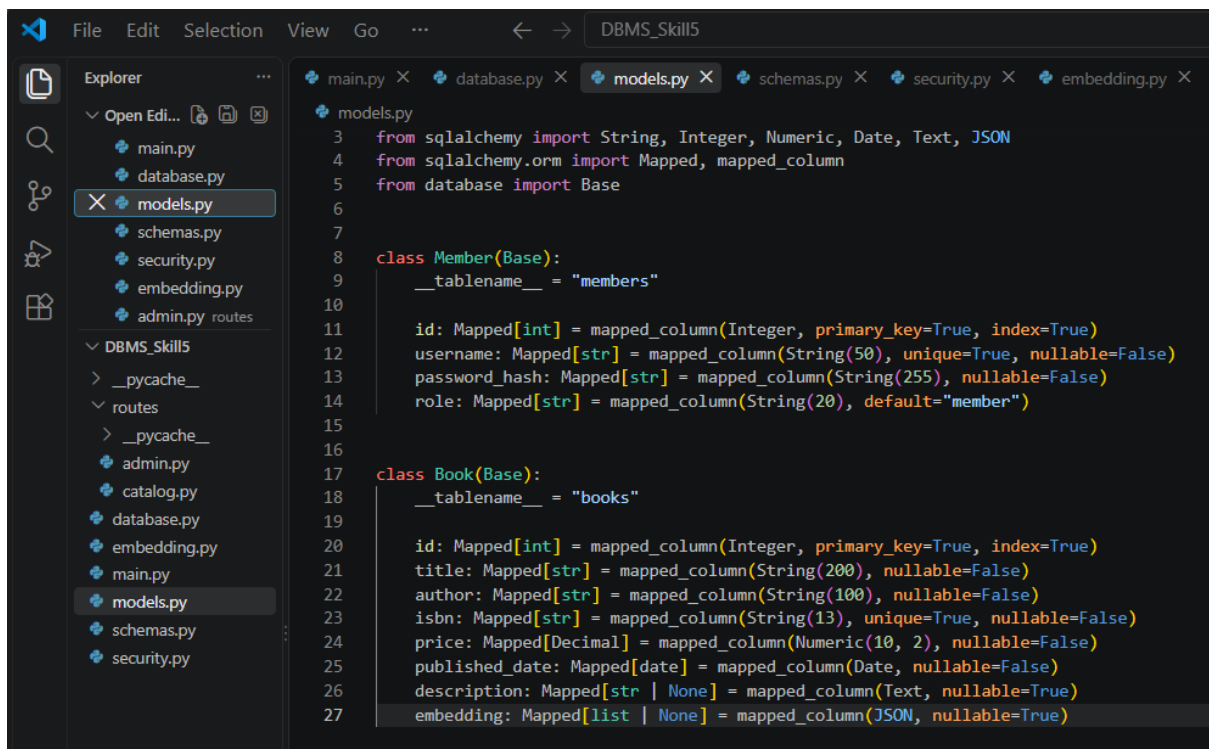


Practical Week 5(DBMS)

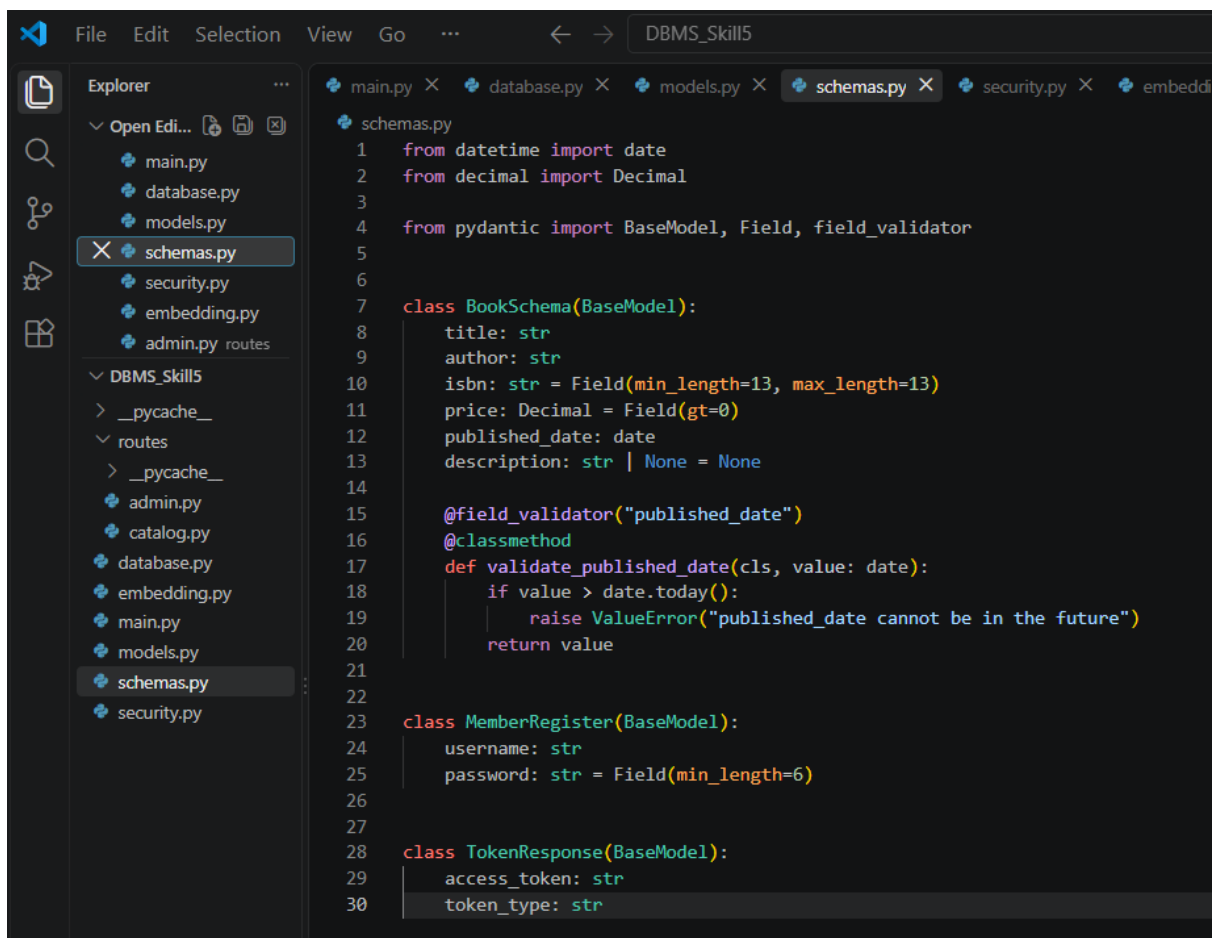


```
Problems  Output  Debug Console  Terminal  Ports

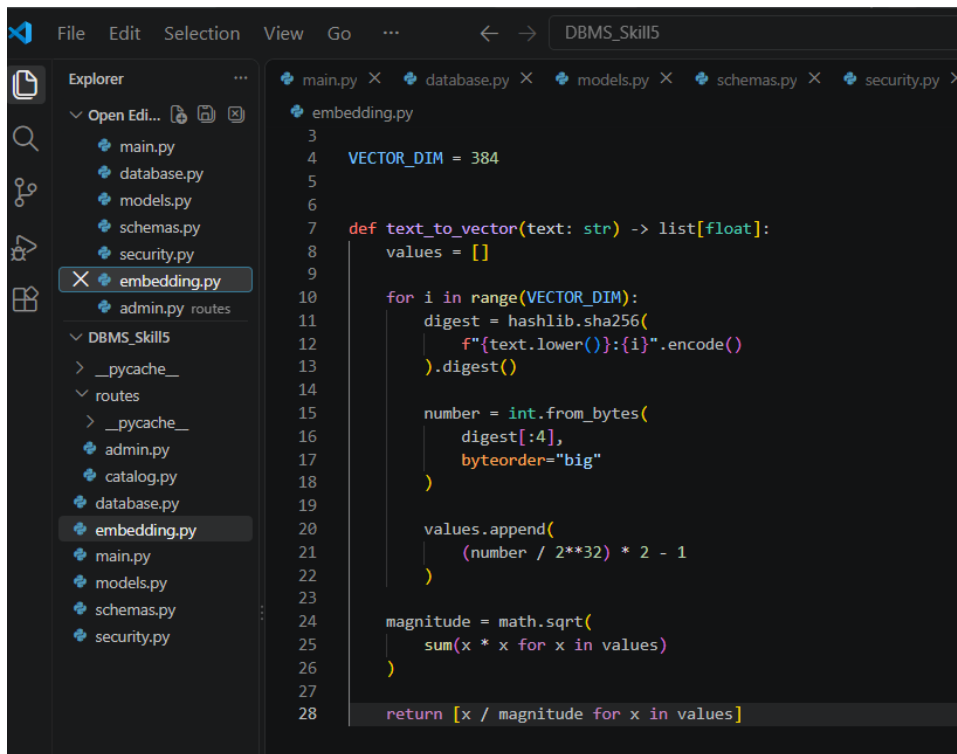
PS C:\Users\koumu\OneDrive\Desktop\DBMS_Skill15> python -m pip install python-jose[cryptography]
Requirement already satisfied: cffi>=2.0.0 in C:\Python314\Lib\site-packages (from cryptography>=3.4.0->python-jose[cryptography]) (2.1.1)
Requirement already satisfied: pycparser in C:\Python314\Lib\site-packages (from cffi>=2.0.0->cryptography>=3.4.0->python-jose[cryptography]) (3.0)
Requirement already satisfied: six>=1.9.0 in C:\Python314\Lib\site-packages (from ecdsa!=0.15->python-jose[cryptography]) (1.17.0)
• PS C:\Users\koumu\OneDrive\Desktop\DBMS_Skill15> python -m pip install passlib[bcrypt] python-multipart
Requirement already satisfied: python-multipart in C:\Python314\Lib\site-packages (0.0.32)
Requirement already satisfied: passlib[bcrypt] in C:\Python314\Lib\site-packages (1.7.4)
Requirement already satisfied: bcrypt>=3.1.0 in C:\Python314\Lib\site-packages (from passlib[bcrypt]) (5.0.0)
○ PS C:\Users\koumu\OneDrive\Desktop\DBMS_Skill15> python -m uvicorn main:app --reload
```



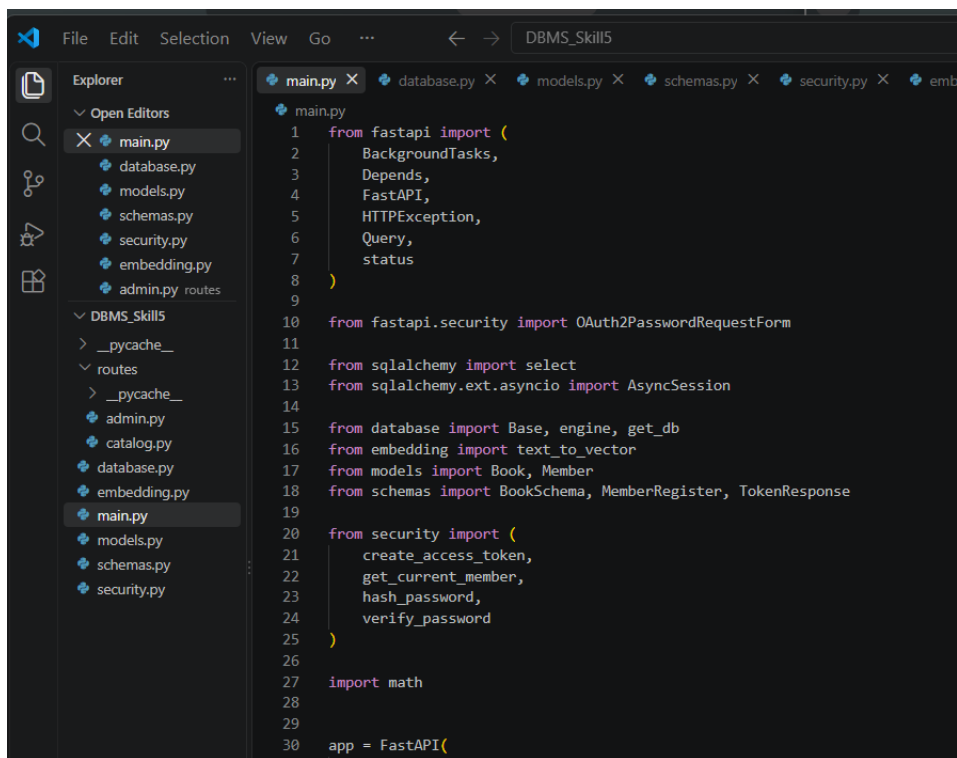
```
1 from sqlalchemy import String, Integer, Numeric, Date, Text, JSON
2 from sqlalchemy.orm import Mapped, mapped_column
3 from database import Base
4
5 class Member(Base):
6     __tablename__ = "members"
7
8     id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
9     username: Mapped[str] = mapped_column(String(50), unique=True, nullable=False)
10    password_hash: Mapped[str] = mapped_column(String(255), nullable=False)
11    role: Mapped[str] = mapped_column(String(20), default="member")
12
13 class Book(Base):
14     __tablename__ = "books"
15
16     id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
17     title: Mapped[str] = mapped_column(String(200), nullable=False)
18     author: Mapped[str] = mapped_column(String(100), nullable=False)
19     isbn: Mapped[str] = mapped_column(String(13), unique=True, nullable=False)
20     price: Mapped[Decimal] = mapped_column(Numeric(10, 2), nullable=False)
21     published_date: Mapped[date] = mapped_column(Date, nullable=False)
22     description: Mapped[str | None] = mapped_column(Text, nullable=True)
23     embedding: Mapped[list | None] = mapped_column(JSON, nullable=True)
```



```
1 from datetime import date
2 from decimal import Decimal
3
4 from pydantic import BaseModel, Field, field_validator
5
6 class BookSchema(BaseModel):
7     title: str
8     author: str
9     isbn: str = Field(min_length=13, max_length=13)
10    price: Decimal = Field(gt=0)
11    published_date: date
12    description: str | None = None
13
14    @field_validator("published_date")
15    @classmethod
16    def validate_published_date(cls, value: date):
17        if value > date.today():
18            raise ValueError("published_date cannot be in the future")
19        return value
20
21 class MemberRegister(BaseModel):
22     username: str
23     password: str = Field(min_length=6)
24
25 class TokenResponse(BaseModel):
26     access_token: str
27     token_type: str
```



```
3
4 VECTOR_DIM = 384
5
6
7 def text_to_vector(text: str) -> list[float]:
8     values = []
9
10    for i in range(VECTOR_DIM):
11        digest = hashlib.sha256(
12            f"{text.lower()}:{i}".encode()
13        ).digest()
14
15        number = int.from_bytes(
16            digest[:4],
17            byteorder="big"
18        )
19
20        values.append(
21            (number / 2**32) * 2 - 1
22        )
23
24    magnitude = math.sqrt(
25        sum(x * x for x in values)
26    )
27
28    return [x / magnitude for x in values]
```



```
1 from fastapi import (
2     BackgroundTasks,
3     Depends,
4     FastAPI,
5     HTTPException,
6     Query,
7     status
8 )
9
10 from fastapi.security import OAuth2PasswordRequestForm
11
12 from sqlalchemy import select
13 from sqlalchemy.ext.asyncio import AsyncSession
14
15 from database import Base, engine, get_db
16 from embedding import text_to_vector
17 from models import Book, Member
18 from schemas import BookSchema, MemberRegister, TokenResponse
19
20 from security import (
21     create_access_token,
22     get_current_member,
23     hash_password,
24     verify_password
25 )
26
27 import math
28
29
30 app = FastAPI()
```

The image shows a Visual Studio Code editor window with a dark theme. The Explorer sidebar on the left displays a project structure for 'DBMS_Skill5'. The file 'security.py' is selected and open in the main editor. The code in 'security.py' includes imports for datetime, timedelta, timezone, FastAPI security utilities, JWT, passlib, and SQLAlchemy. It defines a SECRET_KEY, ALGORITHM, and ACCESS_TOKEN_EXPIRE_MINUTES. Functions for password hashing, verification, and access token creation are implemented.

```
1 from datetime import datetime, timedelta, timezone
2
3 from fastapi import Depends, HTTPException
4 from fastapi.security import OAuth2PasswordBearer
5 from jose import JWTError, jwt
6 from passlib.context import CryptContext
7 from sqlalchemy import select
8 from sqlalchemy.ext.asyncio import AsyncSession
9
10 from database import get_db
11 from models import Member
12
13
14 SECRET_KEY = "CHANGE_THIS_SECRET"
15 ALGORITHM = "HS256"
16 ACCESS_TOKEN_EXPIRE_MINUTES = 60
17
18 pwd_context = CryptContext(
19     schemes=["bcrypt"],
20     deprecated="auto"
21 )
22
23 oauth2_scheme = OAuth2PasswordBearer(
24     tokenUrl="/login"
25 )
26
27
28 def hash_password(password: str):
29     return pwd_context.hash(password)
30
31
32 def verify_password(plain_password, hashed_password):
33     return pwd_context.verify(
34         plain_password,
35         hashed_password
36     )
37
38
39 def create_access_token(username, role):
40     expire = datetime.now(timezone.utc) + timedelta(
41         minutes=ACCESS_TOKEN_EXPIRE_MINUTES
42     )
43
44     payload = {
45         "sub": username,
46         "role": role,
47         "exp": expire
48     }
49
50     return jwt.encode(
51         payload,
52         SECRET_KEY,
53         algorithm=ALGORITHM
```

